LAB 6: EXERCISES

Tiến trình và tín hiệu trong Linux

Lập trình hệ thống (UET.CE2020)
Instructor: Pham Van Duc

Ngày 29 tháng 10 năm 2026

Phần 1. Chuẩn bị trước khi thực hành

Tạo thư mục làm việc

```
mkdir -p lab6
cd lab6
```

Tạo tệp dữ liệu

Một bài thực hành sử dụng tệp `Hello.txt`. Sinh viên tự tạo tệp bằng lệnh Linux đã học:

```
echo -n 'Hello! What is your name?' > Hello.txt
cat Hello.txt; echo
```

Chuẩn bị hai cửa sổ terminal

Một số bài cần quan sát tiến trình hoặc gửi tín hiệu từ bên ngoài. Sinh viên nên mở hai cửa sổ terminal cùng trỏ tới thư mục `lab6`.
Các lệnh quan sát thường dùng:

```
ps -o pid,ppid,stat,cmd
pstree -p
kill -TERM <PID>
kill -INT <PID>
```

Lưu ý
PID thay đổi giữa các lần chạy. Kết quả minh họa chỉ thể hiện cấu trúc đầu ra, không yêu cầu PID thực tế phải giống ví dụ.

Phần 2. Bài tập thực hành

Bài 1. Quan sát PID và PPID

Viết chương trình C `bai1.c` để quan sát thông tin định danh của tiến trình đang chạy.

1. In PID của tiến trình bằng `getpid()` và PPID bằng `getppid()`.

2. Giữ chương trình tiếp tục chạy trong 10 giây bằng `sleep(10)` trước khi kết thúc.
3. Trong terminal thứ hai, dùng `ps` để tìm tiến trình và đối chiếu PID, PPID với kết quả của chương trình.
4. Dùng `pstree -p` để xác định tiến trình trong cây tiến trình.

Kết quả (ví dụ):

```
PID = 4210
PPID = 3982
```

Bài 2. Tạo tiến trình con bằng `fork()`

Viết chương trình C `bai2.c` sử dụng `fork()` để tạo một tiến trình con.

1. Kiểm tra đầy đủ ba trường hợp trả về của `fork()`: Lỗi, tiến trình con và tiến trình cha.
2. Trong tiến trình con, in PID và PPID của tiến trình con.
3. Trong tiến trình cha, in PID của tiến trình cha và PID của tiến trình con do `fork()` trả về.
4. Chạy chương trình nhiều lần và quan sát thứ tự xuất hiện của hai dòng kết quả.

Kết quả: Thứ tự hai dòng có thể thay đổi giữa các lần chạy. Ví dụ:

```
Parent: PID=4300 child=4301
Child: PID=4301 PPID=4300
```

Bài 3. So sánh trường hợp có và không có `waitpid()`

Viết hai chương trình C `bai3a.c` và `bai3b.c` để so sánh thứ tự thực thi khi tiến trình cha có và không chờ tiến trình con.

1. Trong cả hai chương trình, dùng `fork()` để tạo một tiến trình con, tiến trình con in `I am the child`, tiến trình cha in `I am the parent`.
2. Trong `bai3a.c`, không sử dụng `wait()` hoặc `waitpid()` trước khi tiến trình cha in thông báo.
3. Trong `bai3b.c`, tiến trình cha gọi `waitpid()` để chờ tiến trình con kết thúc rồi mới in thông báo.
4. Chạy mỗi chương trình ít nhất ba lần và so sánh thứ tự hai thông báo.

Kết luận cần rút ra: Không có `waitpid()`, chương trình không được giả định cha hoặc con luôn chạy trước. Khi cha gọi `waitpid()`, phần mã của cha đặt sau lời gọi này chỉ tiếp tục sau khi tiến trình con đã kết thúc.

Bài 4. Kiểm tra dữ liệu sau `fork()`

Viết chương trình C `bai4.c` để kiểm tra sự độc lập của không gian địa chỉ giữa tiến trình cha và tiến trình con.

1. Khai báo biến toàn cục `static int idata = 1000` và biến cục bộ `int istack = 150` trước khi gọi `fork()`.
2. Trong tiến trình con, thay đổi `idata` và `istack` sang hai giá trị khác rồi in các giá trị mới.
3. Trong tiến trình cha, gọi `waitpid()` để chờ con kết thúc rồi in lại `idata` và `istack`.
4. Viết phần nhận xét ngắn giải thích vì sao thay đổi của tiến trình con không làm thay đổi giá trị mà tiến trình cha quan sát được.

Kết quả cần quan sát: Hai tiến trình bắt đầu với cùng giá trị của các biến tại thời điểm `fork()`, nhưng sau đó làm việc trên các không gian địa chỉ riêng.

Bài 5. Quan sát vị trí tệp dùng chung sau `fork()`

Viết chương trình C `bai5.c` để kiểm tra sự thay đổi vị trí đọc của tệp khi file descriptor được kế thừa qua `fork()`.

1. Mở `Hello.txt` bằng `open()` với cờ `O_RDONLY` trước khi gọi `fork()`.
2. Trong tiến trình con, dùng `lseek(fd, 7, SEEK_SET)` để thay đổi vị trí đọc rồi kết thúc.
3. Tiến trình cha dùng `waitpid()` chờ con kết thúc.
4. Tiến trình cha dùng chính `fd` đã mở trước `fork()` để đọc phần còn lại của tệp và in ra màn hình.
5. Giải thích vì sao tiến trình cha bắt đầu đọc từ vị trí mà tiến trình con đã đặt.

Kết quả: Với nội dung chuẩn bị ở đầu tài liệu, phần tiến trình cha đọc được bắt đầu từ ký tự `W`:

```
What is your name?
```

Điểm cần nhớ
Sau `fork()`, cha và con có bảng file descriptor riêng, nhưng các descriptor kế thừa từ cùng một lần `open()` cùng tham chiếu tới một *open file description* trong kernel. Vì vậy, chúng có thể quan sát cùng file offset.

Bài 6. Đọc trạng thái kết thúc bằng `waitpid()`

Viết chương trình C `bai6.c` để tiến trình cha nhận và giải mã trạng thái kết thúc của tiến trình con.

1. Tiến trình con kết thúc bằng `_exit(7)`.
2. Tiến trình cha gọi `waitpid()` để nhận biến `status`.
3. Dùng `WIFEXITED(status)` để kiểm tra tiến trình con có kết thúc bình thường hay không.

4. Dùng `WEXITSTATUS(status)` để in mã kết thúc của tiến trình con.

Kết quả (ví dụ):

```
Child exit code = 7
```

Bài 7. Chạy lệnh `ls` bằng họ hàm `exec`

Viết chương trình C `bai7.c` để tiến trình con thực thi lệnh `ls -l` bằng một hàm thuộc họ `exec`.

1. Tạo tiến trình con bằng `fork()`.
2. Trong tiến trình con, dùng `execlp()` hoặc `execvp()` để thực thi lệnh `ls -l`.
3. Nếu lời gọi `exec` thất bại, dùng `perror()` rồi kết thúc tiến trình con bằng `_exit(127)`.
4. Tiến trình cha dùng `waitpid()` để chờ tiến trình con kết thúc.

Kết quả: Chương trình hiển thị kết quả tương tự khi chạy trực tiếp lệnh tương đương:

```
ls -l
```

Bài 8. Xử lý trường hợp `exec()` thất bại

Viết chương trình C `bai8.c` để chủ động tạo trường hợp `execvp()` thất bại và kiểm tra cách xử lý lỗi.

1. Tạo tiến trình con bằng `fork()`.
2. Trong tiến trình con, gọi `execvp()` với một tên chương trình không tồn tại, ví dụ `not_a_command`.
3. Ngay sau `execvp()`, gọi `perror("execvp")` rồi `_exit(127)`.
4. Tiến trình cha dùng `waitpid()` và in mã kết thúc của con.
5. Giải thích vì sao đoạn mã sau `execvp()` chỉ chạy khi lời gọi này thất bại.

Kết quả cần quan sát: Tiến trình con báo lỗi từ `execvp()`, còn tiến trình cha nhận được mã kết thúc 127.

Bài 9. Quan sát tiến trình zombie

Viết chương trình C `bai9.c` để tạo có chủ đích một tiến trình zombie trong thời gian ngắn và quan sát trạng thái của nó bằng `ps`.

1. Tiến trình con kết thúc ngay bằng `_exit(0)`.
2. Tiến trình cha in PID của con rồi ngủ 20 giây mà chưa gọi `waitpid()`.
3. Trong terminal thứ hai, dùng `ps -o pid,ppid,stat,cmd -p <PID>` để quan sát trạng thái `Z`.
4. Sau 20 giây, tiến trình cha gọi `waitpid()` để thu thập tiến trình con trước khi kết thúc.

Điểm cần quan sát: Trong khoảng thời gian tiến trình cha đang ngủ, tiến trình con đã kết thúc nhưng chưa được thu thập nên có thể xuất hiện ở trạng thái zombie.

Bài 10. Quan sát tiến trình orphan

Viết chương trình C `bai10.c` để tạo có chủ đích một tiến trình orphan và quan sát sự thay đổi PPID của tiến trình con.

1. Sau `fork()`, tiến trình cha in PID rồi kết thúc trước tiến trình con.
2. Tiến trình con in PPID ban đầu, ngủ 3 giây rồi in lại PPID.
3. So sánh hai giá trị PPID của tiến trình con.
4. Dùng `ps` hoặc `pstree -p` để quan sát quan hệ cha – con sau khi tiến trình cha ban đầu đã kết thúc.

Kết quả cần giải thích: PPID của tiến trình con có thể thay đổi vì hệ thống gán tiến trình orphan cho một tiến trình giám sát khác. Không yêu cầu PPID mới phải luôn bằng 1.

Bài 11. Xử lý `SIGINT`

Viết chương trình C `bai11.c` bắt tín hiệu `SIGINT` khi người dùng nhấn `Ctrl+C` và kết thúc chương trình theo cách có kiểm soát.

1. Khai báo biến cờ kiểu `static volatile sig_atomic_t stop = 0`.
2. Đăng ký hàm xử lý `SIGINT` bằng `sigaction()`, trong đó handler chỉ gán `stop = 1`.
3. Chương trình in PID rồi tiếp tục chạy cho đến khi người dùng nhấn `Ctrl+C`.
4. Sau khi cờ dừng được đặt, chương trình in `SIGINT received, exiting` rồi kết thúc bình thường.

Lưu ý
Không gọi `printf()` trực tiếp trong signal handler.

Bài 12. Kết thúc có kiểm soát bằng `SIGTERM`

Viết chương trình C `bai12.c` xử lý `SIGTERM` bằng `sigaction()` và kết thúc chương trình theo cách có kiểm soát.

1. Đăng ký handler cho `SIGTERM` bằng `sigaction()`, trong đó handler chỉ đặt cờ dừng.
2. Chương trình in PID rồi tiếp tục chờ cho đến khi cờ dừng được đặt.
3. Từ terminal thứ hai, gửi tín hiệu bằng lệnh `kill -TERM <PID>`.
4. Sau khi nhận tín hiệu, chương trình in `Clean shutdown` rồi kết thúc bình thường.

Kết quả (ví dụ):

```
Terminal 1:
PID = 5102
Clean shutdown

Terminal 2:
kill -TERM 5102
```

Phần 3. Bài tập mở rộng

Các bài dưới đây thực hiện sau khi hoàn thành các bài chính.

Mở rộng 1. Vùng đệm stdio và `fork()`

Viết chương trình C để quan sát ảnh hưởng của vùng đệm stdio khi gọi `fork()`.

1. Viết `printf("Before")` không có ký tự xuống dòng trước khi gọi `fork()`.
2. Sau `fork()`, cả cha và con đều kết thúc bằng `return 0` hoặc `exit()`.
3. Chạy trực tiếp trên terminal và chạy lại với chuyển hướng `./program > out.txt`.
4. Quan sát số lần chuỗi `Before` xuất hiện và giải thích vai trò của vùng đệm stdio.
5. Thử thêm `fflush(stdout)` trước `fork()` và so sánh kết quả.

Mở rộng 2. Chờ không chặn với `WNOHANG`

Viết chương trình C sử dụng `waitpid()` với tùy chọn `WNOHANG` để kiểm tra tiến trình con mà không chặn tiến trình cha.

1. Tiến trình con ngủ 5 giây rồi kết thúc.
2. Tiến trình cha mỗi giây gọi `waitpid(pid, &status, WNOHANG)`.
3. Nếu giá trị trả về là 0, in `Child is still running`.
4. Khi `waitpid()` trả về PID của con, in trạng thái kết thúc và dừng vòng lặp.

Mở rộng 3. Thu thập tiến trình con khi nhận `SIGCHLD`

Viết chương trình C tạo hai tiến trình con và sử dụng `SIGCHLD` để phát hiện khi tiến trình con thay đổi trạng thái.

1. Tạo hai tiến trình con có thời gian ngủ khác nhau.
2. Đăng ký handler cho `SIGCHLD`, trong đó handler chỉ đặt một cờ kiểu `sig_atomic_t`.
3. Trong luồng chính, khi cờ được đặt, lặp `waitpid(-1, &status, WNOHANG)` để thu thập tất cả tiến trình con đã kết thúc.
4. Kết thúc chương trình sau khi đã thu thập đủ hai tiến trình con.

Lưu ý
Không giả định một lần nhận `SIGCHLD` luôn tương ứng với đúng một tiến trình con.

Phần 4. Yêu cầu trước khi hoàn thành Lab

Trước khi kết thúc buổi thực hành, sinh viên cần kiểm tra:

- Mọi lời gọi `fork()`, `exec()`, `waitpid()` và `sigaction()` quan trọng đều được kiểm tra giá trị trả về.
- Không giả định thứ tự cha – con nếu chương trình chưa có cơ chế đồng bộ.
- Mã sau lời gọi `exec()` chỉ dùng để xử lý trường hợp lời gọi thất bại.
- Dùng các macro `WIFEXITED`, `WEXITSTATUS`, `WIFSIGNALED` và `WTERMSIG` khi đọc trạng thái kết thúc.
- Không để zombie tồn tại ngoài thời gian cần thiết cho bài quan sát.
- Signal handler chỉ thực hiện công việc tối thiểu, ưu tiên đặt cờ và xử lý phần còn lại trong luồng chính.
- Chương trình biên dịch không có cảnh báo quan trọng với `-Wall -Wextra`.